

# LangChain & LangGraph

## 8-Week Comprehensive Study Plan

Intermediate Python

No prior LLM exp

3-4 hrs / day

~50 days total

### Week 1

LLM Foundations — APIs, Embeddings, RAG, Agents

### Weeks 2-3

LangChain Core + Advanced — LCEL, Memory, Tools, Routing

### Weeks 4-5

LangGraph — StateGraph, Agents, Checkpoints, Multi-Agent

### Weeks 6-8

Real Projects + Production — FastAPI, Evaluation, Testing

### HOW TO USE THIS PLAN

Each day has the following sections in order:

① Theory → ② Focus/Skip Guide → ③ Pre-reading Resources → ④ Practical Tasks → ⑤ Task Resources → ⑥ Code Examples → ⑦ Topic References → ⑧ Day Completion Checklist

Tick off the checklist items as you complete them. Days build on each other — don't skip.

# Quick Reference

## Week 1:

```
pip install openai python-dotenv chromadb pypdf
```

## Week 2:

```
pip install langchain langchain-openai langchain-community langchain-chroma
```

## Week 3:

```
pip install langsmith tavily-python
```

## Week 4:

```
pip install langgraph
```

## Week 7:

```
pip install fastapi uvicorn httpx
```

## Week 8:

```
pip install pytest ragas datasets
```

## Recommended order if short on time

- ☐ Days 1–5 are non-negotiable — do these before any framework
- ☐ Day 4 (RAG concept) must come before LangChain Week 2
- ☐ Skip Days 6–7 project to reach LangChain 4 days faster
- ☐ Weeks 4–5 (LangGraph) require Weeks 2–3 to make sense — don't skip
- ☐ Weeks 6–8 projects are where real understanding solidifies

# Week 1 — LLM Foundations

## Day 1 · How LLMs Work + API Setup

### ① THEORY — TOPICS TO COVER

- What is a Large Language Model — tokens, context windows, temperature
- Difference between base models vs instruction-tuned models
- Prompts, completions, and the chat message format
- OpenAI API overview: endpoints, authentication, pricing basics

### ② FOCUS / SKIP GUIDE

#### ◆ FOCUS ON

- What tokens are and how text gets split into them
- How context windows limit what the model can 'see'
- What temperature does to randomness in outputs
- How the chat message format (system / user / assistant) works
- Base vs instruction-tuned models — why this matters for prompting

#### ✗ DON'T WORRY ABOUT YET

- Exact transformer architecture internals and attention math
- How the model was trained on the pre-training corpus
- Tokenizer vocabulary design and BPE encoding details
- RLHF fine-tuning pipeline specifics

### RESOURCES — WATCH / READ BEFORE STARTING

- [YouTube]** 3Blue1Brown — But what is a GPT? (44 min)  
→ <https://www.youtube.com/watch?v=wjZofJX0v4M>
- [YouTube]** Andrej Karpathy — Intro to LLMs (1 hr)  
→ [https://www.youtube.com/watch?v=zjkBMFhNj\\_g](https://www.youtube.com/watch?v=zjkBMFhNj_g)
- [Docs]** OpenAI Quickstart — official API docs  
→ <https://platform.openai.com/docs/quickstart>
- [Blog]** Lena Voita — NLP Course: Language Models  
→ [https://lena-voita.github.io/nlp\\_course/language\\_modeling.html](https://lena-voita.github.io/nlp_course/language_modeling.html)
- [Blog]** Simon Willison — How LLMs work explained simply  
→ <https://simonwillison.net/2023/Aug/3/weird-world-of-llms/>

### ③ PRACTICAL — TASKS TO COMPLETE

- ☐ Install Python env: pip install openai python-dotenv
- ☐ Set up .env file and load API key with python-dotenv
- ☐ Write a script calling gpt-3.5-turbo and print a completion
- ☐ Change temperature (0 vs 1) and compare 5 outputs side by side

## TASK RESOURCES

- [Docs]** OpenAI Python SDK on GitHub  
→ <https://github.com/openai/openai-python>
- [Docs]** python-dotenv docs  
→ <https://pypi.org/project/python-dotenv/>
- [Blog]** Real Python — Working with .env files  
→ <https://realpython.com/python-dotenv/>

## ④ CODE EXAMPLES

### Task 1-2: Setup .env + install

```
# Terminal
pip install openai python-dotenv

# .env file (create in project root)
OPENAI_API_KEY=sk-...
```

### Task 3: First API call

```
import os
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))

response = client.chat.completions.create(
    model='gpt-3.5-turbo',
    messages=[{'role': 'user', 'content': 'Explain tokens in 2 sentences.'}]
)
print(response.choices[0].message.content)
```

### Task 4: Compare temperature values

```
for temp in [0, 0.5, 1.0]:
    resp = client.chat.completions.create(
        model='gpt-3.5-turbo',
        temperature=temp,
        messages=[{'role': 'user', 'content': 'Give me a creative name for a coffee shop.'}]
    )
    print(f'temp={temp}: {resp.choices[0].message.content}')
```

## READ MORE — TOPIC REFERENCES

- [Blog]** Jay Alammar — The Illustrated GPT-2  
→ <https://jalammar.github.io/illustrated-gpt2/>
- [Blog]** Chip Huyen — LLM Applications Overview  
→ <https://huyenchip.com/2023/04/11/llm-engineering.html>

## ⑤ DAY COMPLETION CHECKLIST

- ☐ Watched 3Blue1Brown GPT video and understood the token concept
- ☐ Watched Karpathy LLM intro video
- ☐ openai and python-dotenv installed without errors

- ☐ .env file created with OPENAI\_API\_KEY loaded correctly
- ☐ First API call script runs and prints a completion
- ☐ Temperature comparison: noted differences between temp=0 and temp=1
- ☐ Can explain: what is a token, what is a context window, what is temperature

## Day 2 · Prompt Engineering Fundamentals

### ① THEORY — TOPICS TO COVER

- System vs user vs assistant message roles
- Zero-shot, one-shot, few-shot prompting patterns
- Chain-of-thought prompting technique
- JSON mode and structured outputs from the API

### ② FOCUS / SKIP GUIDE

#### ◆ FOCUS ON

- How the system / user / assistant roles shape model behavior
- When to use zero-shot vs few-shot — and how to write good examples
- Chain-of-thought: adding 'think step by step' and why it helps
- Requesting JSON output reliably — json\_object response format
- Building and maintaining a multi-turn messages list manually

#### ✗ DON'T WORRY ABOUT YET

- Automatic prompt optimisation tools (DSPy, TextGrad)
- Meta-prompting and self-consistency techniques
- Fine-tuning as an alternative to prompting — save for later

### RESOURCES — WATCH / READ BEFORE STARTING

- [Course] Andrew Ng — ChatGPT Prompt Engineering for Developers (DeepLearning.AI)  
→ <https://www.deeplearning.ai/short-courses/chatgpt-prompt-engineering-for-developers/>
- [Docs] promptingguide.ai — comprehensive prompting reference  
→ <https://www.promptingguide.ai/>
- [Docs] OpenAI — Prompt engineering best practices  
→ <https://platform.openai.com/docs/guides/prompt-engineering>
- [Blog] Lil'Log — Prompt Engineering  
→ <https://lilianweng.github.io/posts/2023-03-15-prompt-engineering/>

### ③ PRACTICAL — TASKS TO COMPLETE

- ☐ Build a multi-turn chat loop using the messages list pattern
- ☐ Write a few-shot prompt that extracts structured data from text
- ☐ Use JSON mode — make the model return consistent JSON
- ☐ Create a system prompt persona (e.g. a helpful data analyst)

## TASK RESOURCES

**[Docs]** OpenAI — Structured outputs guide

→ <https://platform.openai.com/docs/guides/structured-outputs>

**[Blog]** Real Python — Python while loops for chat interfaces

→ <https://realpython.com/python-while-loop/>

## ④ CODE EXAMPLES

### Task 1: Multi-turn chat loop

```
messages = [{'role': 'system', 'content': 'You are a helpful assistant.'}]

while True:
    user_input = input('You: ')
    if user_input.lower() == 'quit': break
    messages.append({'role': 'user', 'content': user_input})
    response = client.chat.completions.create(
        model='gpt-3.5-turbo', messages=messages
    )
    reply = response.choices[0].message.content
    messages.append({'role': 'assistant', 'content': reply})
    print(f'Assistant: {reply}')
```

### Task 2: Few-shot structured extraction

```
few_shot_system = '''Extract name, age, city as JSON.
Example: 'John is 30, from NYC' -> {"name":"John","age":30,"city":"NYC"}
Example: 'Sara, 25, lives in London' -> {"name":"Sara","age":25,"city":"London"}
'''

response = client.chat.completions.create(
    model='gpt-3.5-turbo',
    response_format={'type': 'json_object'},
    messages=[
        {'role': 'system', 'content': few_shot_system},
        {'role': 'user', 'content': 'Mike is 42 and lives in Berlin'}
    ]
)
import json
data = json.loads(response.choices[0].message.content)
print(data) # {'name': 'Mike', 'age': 42, 'city': 'Berlin'}
```

## READ MORE — TOPIC REFERENCES

**[Blog]** Wei et al. — Chain-of-Thought Prompting paper (2022)

→ <https://arxiv.org/abs/2201.11903>

**[Blog]** Learn Prompting — free online guide

→ <https://learnprompting.org/>

## ⑤ DAY COMPLETION CHECKLIST

- ☐ Understand the difference between system, user, and assistant roles
- ☐ Built a working multi-turn chat loop that maintains history
- ☐ Written a few-shot prompt that reliably extracts structured data
- ☐ Made the API return valid JSON using `response_format json_object`

- ☐ Created a system persona prompt and tested how it changes responses
- ☐ Can explain chain-of-thought and demonstrated it improves a math answer

## Day 3 · Embeddings & Vector Similarity

### ① THEORY — TOPICS TO COVER

- What embeddings are — turning text into number vectors
- Cosine similarity and why it matters for semantic search
- Embedding models: text-embedding-3-small, ada-002
- Intro to vector stores: FAISS, Chroma, Pinecone (concepts)

### ② FOCUS / SKIP GUIDE

#### ◆ FOCUS ON

- How embeddings encode meaning — similar texts → similar vectors
- What a vector space looks like and why distance matters
- How cosine similarity is calculated (dot product / magnitudes)
- How to call the OpenAI Embeddings API and inspect vector shape
- How ChromaDB stores and queries vectors locally

#### ✗ DON'T WORRY ABOUT YET

- How the embedding model was trained (contrastive learning details)
- Dimensionality reduction techniques (PCA, UMAP) for now
- Pinecone, Weaviate, Qdrant — come back to these in Week 2
- Fine-tuning embedding models on custom data

### RESOURCES — WATCH / READ BEFORE STARTING

- [Blog] Vicki Boykis — What are embeddings? (deep visual guide)  
→ [https://vickiboykis.com/what\\_are\\_embeddings/](https://vickiboykis.com/what_are_embeddings/)
- [Docs] OpenAI Embeddings guide  
→ <https://platform.openai.com/docs/guides/embeddings>
- [Docs] ChromaDB quickstart  
→ <https://docs.trychroma.com/getting-started>
- [YouTube] Sentdex — Word Embeddings explanation (15 min)  
→ <https://www.youtube.com/watch?v=5PL0TmQhItY>

### ③ PRACTICAL — TASKS TO COMPLETE

- ☐ Call OpenAI Embeddings API and inspect the returned vector
- ☐ Calculate cosine similarity between two sentences manually
- ☐ Build semantic search: embed 10 sentences, find closest to query
- ☐ Install chromadb and store/query 20 text chunks

## TASK RESOURCES

[Docs] numpy dot product docs

→ <https://numpy.org/doc/stable/reference/generated/numpy.dot.html>

[Docs] ChromaDB Python client API

→ <https://docs.trychroma.com/reference/py-client>

[Blog] Pinecone — Vector similarity explained

→ <https://www.pinecone.io/learn/vector-similarity/>

## ④ CODE EXAMPLES

### Task 1: Call Embeddings API

```
import numpy as np

response = client.embeddings.create(
    model='text-embedding-3-small',
    input='The cat sat on the mat'
)
vector = response.data[0].embedding
print(f'Vector dimensions: {len(vector)}') # 1536
print(f'First 5 values: {vector[:5]}')
```

### Task 2: Cosine similarity from scratch

```
def cosine_similarity(v1, v2):
    v1, v2 = np.array(v1), np.array(v2)
    return np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2))

def get_embedding(text):
    resp = client.embeddings.create(model='text-embedding-3-small', input=text)
    return resp.data[0].embedding

e1 = get_embedding('I love programming')
e2 = get_embedding('Coding is my passion')
e3 = get_embedding('The weather is nice today')
print(cosine_similarity(e1, e2)) # high ~0.88
print(cosine_similarity(e1, e3)) # low ~0.21
```

### Task 4: ChromaDB store and query

```
import chromadb

client_chroma = chromadb.Client()
collection = client_chroma.create_collection('my_docs')

sentences = ['Python is great for data science',
             'JavaScript runs in the browser',
             # ... add 18 more
             ]
collection.add(
    documents=sentences,
    ids=[f'doc_{i}' for i in range(len(sentences))]
)
results = collection.query(query_texts=['best language for ML'], n_results=3)
print(results['documents'])
```



### READ MORE — TOPIC REFERENCES

**[Blog]** Jay Alammar — The Illustrated Word2Vec

→ <https://jalammar.github.io/illustrated-word2vec/>

**[Blog]** Pinecone — Vector database explained

→ <https://www.pinecone.io/learn/vector-database/>

### ⑤ DAY COMPLETION CHECKLIST

- ☐ Watched embeddings explanation video
- ☐ Called OpenAI Embeddings API and printed vector shape (1536 dims)
- ☐ Calculated cosine similarity manually with numpy — got sensible results
- ☐ Built semantic search over 10 sentences — top result makes sense
- ☐ chromadb installed and collection created, add + query working
- ☐ Can explain: embedding, cosine similarity, vector store

## Day 4 · RAG Concept & Pipeline

### ① THEORY — TOPICS TO COVER

- Why RAG exists — knowledge cutoffs and hallucination
- The RAG pipeline: chunk → embed → store → retrieve → generate
- Text chunking strategies: fixed-size, sentence, paragraph
- Context windows and why we must chunk long documents

### ② FOCUS / SKIP GUIDE

#### ◆ FOCUS ON

- The full RAG pipeline end-to-end and why each step exists
- How chunk size and overlap affect retrieval quality
- How retrieved chunks get injected into the prompt as context
- Why RAG reduces hallucinations compared to plain LLM calls
- How to write a retrieve() function returning top-k chunks

#### ✗ DON'T WORRY ABOUT YET

- Hybrid search combining dense + sparse (BM25) — that's Week 3
- Re-ranking retrieved results — advanced topic for later
- RAG evaluation metrics (faithfulness, context recall) — Week 3
- Parent-document retrieval and HyDE — Week 3 advanced patterns

### RESOURCES — WATCH / READ BEFORE STARTING

**[YouTube]** James Briggs — RAG from scratch in Python (30 min)

→ <https://www.youtube.com/watch?v=sVcwVQRHlc8>

**[YouTube]** IBM — What is RAG? (5 min overview)

→ <https://www.youtube.com/watch?v=T-D1OfcDW1M>

**[Docs]** LangChain RAG conceptual guide

→ <https://python.langchain.com/docs/concepts/rag/>

**[Blog]** Pinecone — RAG explained

→ <https://www.pinecone.io/learn/retrieval-augmented-generation/>

### ③ PRACTICAL — TASKS TO COMPLETE

- ☐ Chunk a text file into 500-char pieces with 50-char overlap
- ☐ Embed all chunks and store in ChromaDB
- ☐ Write a retrieve() function returning top-3 similar chunks
- ☐ Pass retrieved chunks as context to GPT and get a grounded answer

#### TASK RESOURCES

**[Docs]** ChromaDB query docs

→ <https://docs.trychroma.com/usage-guide#querying-a-collection>

**[Blog]** LangChain — Text splitting guide

→ [https://python.langchain.com/docs/how\\_to/split\\_by\\_character/](https://python.langchain.com/docs/how_to/split_by_character/)

### ④ CODE EXAMPLES

#### Task 1: Chunking with overlap

```
def chunk_text(text, chunk_size=500, overlap=50):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start += chunk_size - overlap
    return chunks

with open('my_document.txt') as f:
    text = f.read()
chunks = chunk_text(text)
print(f'Created {len(chunks)} chunks')
```

#### Task 3-4: Retrieve and generate

```
def retrieve(query, collection, n=3):
    results = collection.query(query_texts=[query], n_results=n)
    return results['documents'][0]

def rag_answer(query, collection):
    context_chunks = retrieve(query, collection)
    context = '\n\n'.join(context_chunks)
    messages = [
        {'role': 'system', 'content': 'Answer using ONLY the context below.'},
        {'role': 'user', 'content': f'Context:\n{context}\n\nQuestion: {query}'}
    ]
    resp = client.chat.completions.create(
        model='gpt-3.5-turbo', messages=messages
    )
    return resp.choices[0].message.content
```

#### READ MORE — TOPIC REFERENCES

**[Blog]** Anthropic — Contextual retrieval technique

→ <https://www.anthropic.com/news/contextual-retrieval>

**[Paper]** Lewis et al. — RAG original paper (2020)

→ <https://arxiv.org/abs/2005.11401>

## ⑤ DAY COMPLETION CHECKLIST

- ☐ Watched RAG from scratch video
- ☐ Chunking function works — correctly creates overlapping chunks
- ☐ All chunks embedded and stored in ChromaDB
- ☐ retrieve() function returns top-3 relevant chunks for a query
- ☐ Full RAG pipeline works — model answers using only provided context
- ☐ Tested with 5 questions — model stays grounded in context
- ☐ Can draw the full RAG pipeline from memory: chunk→embed→store→retrieve→generate

## Day 5 · Agents & Tools Concept

### ① THEORY — TOPICS TO COVER

- What is an AI Agent — the perceive, plan, act loop
- What are tools: function calling, external APIs
- ReAct pattern (Reason + Act) for agent decisions
- Chains (fixed path) vs agents (dynamic routing)

### ② FOCUS / SKIP GUIDE

#### ◆ FOCUS ON

- How the OpenAI tools API sends a function call back to your code
- The agent loop: LLM decides → code executes → result returned → repeat
- How to define a tool schema with name, description, parameters
- Why tool descriptions need to be clear — the model reads them to decide
- Difference: a chain always runs the same steps; an agent chooses steps

#### ✗ DON'T WORRY ABOUT YET

- Memory management in long-running agents — that's LangGraph
- Multi-agent coordination and supervisor patterns — Week 5
- Agent evaluation and safety constraints — production topic
- AutoGPT / BabyAGI style recursive agents — complex edge cases

### RESOURCES — WATCH / READ BEFORE STARTING

- [YouTube] AI Explained — What are AI Agents? (15 min)  
→ [https://www.youtube.com/watch?v=Zl\\_O0JrGH08](https://www.youtube.com/watch?v=Zl_O0JrGH08)
- [Docs] OpenAI Function Calling guide  
→ <https://platform.openai.com/docs/guides/function-calling>
- [Docs] LangChain — Agents explained (official)  
→ <https://python.langchain.com/docs/concepts/agents/>
- [Blog] Lil'Log — LLM Powered Autonomous Agents  
→ <https://lilianweng.github.io/posts/2023-06-23-agent/>

### ③ PRACTICAL — TASKS TO COMPLETE

- ☐ Build a basic function-calling example with the OpenAI tools API
- ☐ Create a mock get\_weather tool and have the model call it
- ☐ Build a loop: LLM decides → calls tool → feeds result back

- ☐ Write a mini agent that can add/multiply using tools

### TASK RESOURCES

**[Docs]** OpenAI — Tool use reference

→ <https://platform.openai.com/docs/api-reference/chat/create#chat-create-tools>

**[Blog]** Towards Data Science — Function calling deep dive

→ <https://towardsdatascience.com/function-calling-with-the-openai-api/>

## ④ CODE EXAMPLES

### Task 1-2: Define a tool schema

```
tools = [{
    'type': 'function',
    'function': {
        'name': 'get_weather',
        'description': 'Get current weather for a city',
        'parameters': {
            'type': 'object',
            'properties': {
                'city': {'type': 'string', 'description': 'City name'}
            },
            'required': ['city']
        }
    }
}]
```

### Task 3: Full agent loop

```
import json

def get_weather(city): # mock tool
    return f'Sunny, 25°C in {city}'

def run_agent(user_message):
    messages = [{'role': 'user', 'content': user_message}]
    while True:
        response = client.chat.completions.create(
            model='gpt-3.5-turbo', messages=messages, tools=tools
        )
        msg = response.choices[0].message
        if msg.tool_calls:
            tool_call = msg.tool_calls[0]
            args = json.loads(tool_call.function.arguments)
            result = get_weather(**args)
            messages.append(msg) # assistant turn with tool_call
            messages.append({'role': 'tool', 'tool_call_id': tool_call.id, 'content': result})
        else:
            return msg.content # final answer

print(run_agent('What is the weather in Paris?'))
```

### READ MORE — TOPIC REFERENCES

**[Paper]** Yao et al. — ReAct: Synergizing Reasoning + Acting (2022)

→ <https://arxiv.org/abs/2210.03629>

**[Blog]** Simon Willison — The problem with AI agents

→ <https://simonwillison.net/2023/Apr/25/dual-llm-pattern/>

### ⑤ DAY COMPLETION CHECKLIST

- ☐ Watched AI Agents explanation video
- ☐ Defined a tool schema with correct JSON structure
- ☐ get\_weather mock tool works — model calls it when asked about weather
- ☐ Full agent loop runs — LLM calls tool, gets result, produces final answer
- ☐ Mini calculator agent can add and multiply using tool calls
- ☐ Can explain: what is a tool, what is the agent loop, what is ReAct
- ☐ Understand how chains differ from agents (fixed vs dynamic path)

## Day 6-7 · Week 1 Review + Mini Project

### ① THEORY — TOPICS TO COVER

- Revisit any confusing concepts from Days 1–5
- GPT-3.5 vs GPT-4 vs Claude vs Gemini — key differences
- When to use embeddings vs fine-tuning vs RAG

### ② FOCUS / SKIP GUIDE

#### ◆ FOCUS ON

- Putting all Week 1 concepts together in one working app
- PDF loading, chunking, embedding, storing, retrieval — the full stack
- Building a simple CLI loop that feels like a real Q&A; tool
- Understanding where hallucinations come from in your bot
- Model comparison: capabilities, cost, speed tradeoffs

#### ✗ DON'T WORRY ABOUT YET

- Perfect chunking strategy — naive fixed-size is fine for now
- Evaluation metrics and measuring retrieval quality formally
- Streaming responses — comes in Week 3

### RESOURCES — WATCH / READ BEFORE STARTING

**[Blog]** Real Python — Working with PDFs in Python

→ <https://realpython.com/pdf-python/>

**[YouTube]** Rabbitmetrics — PDF chatbot from scratch

→ <https://www.youtube.com/watch?v=u6tMfAEvCSA>

**[Docs]** pypdf documentation

→ <https://pypdf.readthedocs.io/>

**[Blog]** OpenAI — GPT-4 vs GPT-3.5 comparison

→ <https://openai.com/research/gpt-4>

### ③ PRACTICAL — TASKS TO COMPLETE

- ☐ Build a document Q&A; bot using only the OpenAI API (no LangChain)
- ☐ Load a PDF with pypdf, chunk it, embed it, store in Chroma
- ☐ Add a simple CLI chat interface with a loop
- ☐ Test with 10 questions and note where it fails or hallucinates

#### TASK RESOURCES

**[Docs]** pypdf PdfReader usage

→ <https://pypdf.readthedocs.io/en/stable/user/extract-text.html>

**[Docs]** ChromaDB persistence docs

→ <https://docs.trychroma.com/usage-guide#initiating-a-persistent-chroma-client>

### ④ CODE EXAMPLES

#### Load PDF + full pipeline

```
from pypdf import PdfReader
import chromadb

# 1. Load PDF
reader = PdfReader('my_document.pdf')
full_text = ' '.join(p.extract_text() for p in reader.pages)

# 2. Chunk
chunks = chunk_text(full_text, 500, 50)

# 3. Embed + Store
chroma = chromadb.PersistentClient(path='./chroma_db')
collection = chroma.get_or_create_collection('pdf_docs')
collection.add(documents=chunks, ids=[f'c{i}' for i in range(len(chunks))])

# 4. CLI chat loop
print('PDF loaded. Ask me anything (type quit to exit)')
while True:
    q = input('You: ')
    if q == 'quit': break
    print('Bot:', rag_answer(q, collection))
```

#### READ MORE — TOPIC REFERENCES

**[Blog]** Weights & Biases — LLM comparison guide

→ <https://wandb.ai/onlineinference/mistral/reports/GPT-4-vs-GPT-3-5-vs-Claude-2--Vmlldzo1MjQ1MDk1>

**[Docs]** Anthropic Claude API

→ <https://docs.anthropic.com/en/api/getting-started>

### ⑤ DAY COMPLETION CHECKLIST

- ☐ Reviewed and clarified all Day 1–5 concepts you were unsure about
- ☐ pypdf installed and PDF text extraction working
- ☐ Full pipeline: load → chunk → embed → store in Chroma
- ☐ CLI chat loop works — can ask questions and get responses
- ☐ Tested with 10 questions — logged which ones failed
- ☐ Identified at least one hallucination and understood why it happened

- ☐ Can compare GPT-3.5 vs GPT-4 on cost, capability, speed
  - ☐ Ready to move to Week 2 — LangChain
-

# Week 2 — LangChain Core

## Day 8 · LangChain Intro & LCEL

### ① THEORY — TOPICS TO COVER

- What LangChain is and why it exists (abstraction over LLMs)
- LangChain Expression Language (LCEL) — the | pipe operator
- Core components: LLMs, ChatModels, PromptTemplates, OutputParsers
- RunnableSequence, RunnablePassthrough, RunnableParallel

### ② FOCUS / SKIP GUIDE

#### ◆ FOCUS ON

- The | pipe syntax for chaining components together
- How prompt | llm | parser creates a full chain
- What PromptTemplate does — filling variables into prompt strings
- What OutputParsers do — converting raw text into useful types
- .invoke() vs .stream() — when to use each

#### ✗ DON'T WORRY ABOUT YET

- LangChain v0.1 legacy chains (LLMChain, ConversationChain) — outdated
- LangServe and LangGraph — those come later
- Every Runnable method — focus on invoke, stream, batch for now

### RESOURCES — WATCH / READ BEFORE STARTING

- [Docs]** LangChain LCEL official docs  
→ <https://python.langchain.com/docs/concepts/lcel/>
- [YouTube]** Sam Witteveen — LangChain LCEL deep dive  
→ <https://www.youtube.com/watch?v=O0dUOtOlrfs>
- [Docs]** LangChain — Get started tutorial  
→ <https://python.langchain.com/docs/tutorials/>
- [Blog]** LangChain blog — Why LCEL?  
→ <https://blog.langchain.dev/langchain-expression-language/>

### ③ PRACTICAL — TASKS TO COMPLETE

- ☐ pip install langchain langchain-openai
- ☐ Build your first LCEL chain: prompt | llm | parser
- ☐ Create a PromptTemplate with input variables and invoke it
- ☐ Try .stream() and watch tokens arrive in real time

#### TASK RESOURCES

- [Docs]** LangChain — ChatOpenAI docs  
→ <https://python.langchain.com/docs/integrations/chat/openai/>
- [Docs]** LangChain — PromptTemplate reference  
→ [https://python.langchain.com/docs/concepts/prompt\\_templates/](https://python.langchain.com/docs/concepts/prompt_templates/)

### ④ CODE EXAMPLES



#### First LCEL chain

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(model='gpt-3.5-turbo', temperature=0)
prompt = ChatPromptTemplate.from_template(
    'Explain {topic} in one sentence.'
)
parser = StrOutputParser()

chain = prompt | llm | parser
result = chain.invoke({'topic': 'recursion'})
print(result)

# Streaming
for chunk in chain.stream({'topic': 'machine learning'}):
    print(chunk, end='', flush=True)
```

#### READ MORE — TOPIC REFERENCES

**[Docs]** LangChain — Runnable interface reference  
→ <https://python.langchain.com/docs/concepts/runnables/>  
**[YouTube]** Greg Kamradt — LangChain full course  
→ [https://www.youtube.com/watch?v=\\_v\\_fgW2SkkQ](https://www.youtube.com/watch?v=_v_fgW2SkkQ)

#### ⑤ DAY COMPLETION CHECKLIST

- ☐ langchain and langchain-openai installed without errors
- ☐ First LCEL chain runs: prompt | llm | parser
- ☐ PromptTemplate with input variables works correctly
- ☐ .stream() works — tokens print one by one
- ☐ Understand what each component in the pipe does
- ☐ Can explain LCEL and why it is better than manual chaining

## Day 9 · Prompt Templates & Output Parsers

#### ① THEORY — TOPICS TO COVER

- PromptTemplate vs ChatPromptTemplate — when to use which
- Partial prompt templates and message placeholders
- Output parsers: StrOutputParser, JsonOutputParser, PydanticOutputParser
- Structured output with Pydantic models and format\_instructions

#### ② FOCUS / SKIP GUIDE

### ◆ FOCUS ON

- ChatPromptTemplate with system and human message templates
- MessagesPlaceholder — injecting conversation history dynamically
- PydanticOutputParser — defining exactly what fields you want back
- format\_instructions — how the parser tells the model what to output
- When JSON output is reliable vs when it fails

### ✗ DON'T WORRY ABOUT YET

- FewShotPromptTemplate and example selectors — advanced prompting
- Custom output parsers from BaseOutputParser — not needed yet
- Async output parsers — save for production deployment

### RESOURCES — WATCH / READ BEFORE STARTING

- [Docs] LangChain output parsers docs  
→ [https://python.langchain.com/docs/concepts/output\\_parsers/](https://python.langchain.com/docs/concepts/output_parsers/)
- [YouTube] Greg Kamradt — LangChain structured output  
→ <https://www.youtube.com/watch?v=xZzvwR9jdPA>
- [Docs] Pydantic v2 — Model basics  
→ <https://docs.pydantic.dev/latest/concepts/models/>

### ③ PRACTICAL — TASKS TO COMPLETE

- ☐ Build a chain with ChatPromptTemplate formatting user input
- ☐ Parse LLM output into a Python dict using JsonOutputParser
- ☐ Define a Pydantic model and use PydanticOutputParser
- ☐ Chain: prompt | llm | parser with full format\_instructions flow

### TASK RESOURCES

- [Docs] LangChain — PydanticOutputParser reference  
→ [https://python.langchain.com/docs/how\\_to/output\\_parser\\_pydantic/](https://python.langchain.com/docs/how_to/output_parser_pydantic/)
- [Docs] Pydantic — Field validators  
→ <https://docs.pydantic.dev/latest/concepts/validators/>

### ④ CODE EXAMPLES

#### PydanticOutputParser example

```
from pydantic import BaseModel, Field
from langchain_core.output_parsers import PydanticOutputParser
from langchain_core.prompts import PromptTemplate

class MovieReview(BaseModel):
    title: str = Field(description='Movie title')
    rating: int = Field(description='Rating 1-10')
    summary: str = Field(description='One sentence summary')

parser = PydanticOutputParser(pydantic_object=MovieReview)
prompt = PromptTemplate(
    template='Review this movie: {movie}\n{format_instructions}',
    input_variables=['movie'],
    partial_variables={'format_instructions': parser.get_format_instructions()}
)
chain = prompt | llm | parser
review = chain.invoke({'movie': 'Inception'})
print(review.title, review.rating) # typed access!
```

#### READ MORE — TOPIC REFERENCES

**[Docs]** LangChain — Structured output with `.with_structured_output()`

→ [https://python.langchain.com/docs/how\\_to/structured\\_output/](https://python.langchain.com/docs/how_to/structured_output/)

**[Docs]** LangChain — JsonOutputParser

→ [https://python.langchain.com/docs/how\\_to/output\\_parser\\_json/](https://python.langchain.com/docs/how_to/output_parser_json/)

#### ⑤ DAY COMPLETION CHECKLIST

- ☐ ChatPromptTemplate with system + human messages works
- ☐ JsonOutputParser returns a Python dict correctly
- ☐ Pydantic model defined and PydanticOutputParser hooked up
- ☐ `format_instructions` injected into prompt and model follows the schema
- ☐ Full chain `prompt | llm | parser` runs end-to-end
- ☐ Can access parsed fields as typed attributes (not string parsing)

## Day 10 · Memory & Conversation History

#### ① THEORY — TOPICS TO COVER

- Why LLMs are stateless — you must supply history each call
- ConversationBufferMemory, ConversationSummaryBufferMemory
- Window memory vs summary memory — tradeoffs
- RunnableWithMessageHistory — the LCEL-native approach

#### ② FOCUS / SKIP GUIDE

### ◆ FOCUS ON

- The stateless problem: why you must manually manage messages
- How `RunnableWithMessageHistory` wraps a chain to add memory
- `session_id` — why it enables multiple independent conversations
- When to use buffer memory vs summary memory (token cost tradeoffs)
- How to test that memory actually works across turns

### ✗ DON'T WORRY ABOUT YET

- Database-backed message stores (Redis, Postgres) — production topic
- Summarizing memory with a separate LLM call overhead
- `VectorStoreMemory` — looking up semantically relevant history

### RESOURCES — WATCH / READ BEFORE STARTING

- [Docs] LangChain memory concepts  
→ [https://python.langchain.com/docs/concepts/chat\\_history/](https://python.langchain.com/docs/concepts/chat_history/)
- [YouTube] 1littlecoder — LangChain Memory types  
→ <https://www.youtube.com/watch?v=xChfK3bOVQs>
- [Docs] LangChain — `RunnableWithMessageHistory`  
→ [https://python.langchain.com/docs/how\\_to/message\\_history/](https://python.langchain.com/docs/how_to/message_history/)

### ③ PRACTICAL — TASKS TO COMPLETE

- ☐ Build a CLI chatbot that remembers the last 5 exchanges
- ☐ Implement `RunnableWithMessageHistory` with a chat chain
- ☐ Add summary memory that kicks in after 500 tokens
- ☐ Test: verify earlier context is actually used in later answers

### TASK RESOURCES

- [Docs] LangChain — `InMemoryChatMessageHistory`  
→ [https://python.langchain.com/docs/how\\_to/message\\_history/](https://python.langchain.com/docs/how_to/message_history/)
- [Docs] LangChain — `ConversationTokenBufferMemory`  
→ [https://python.langchain.com/api\\_reference/langchain/memory/langchain.memory.token\\_buffer.ConversationTokenBufferMemory.html](https://python.langchain.com/api_reference/langchain/memory/langchain.memory.token_buffer.ConversationTokenBufferMemory.html)

### ④ CODE EXAMPLES

### RunnableWithMessageHistory

```
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant.'),
    MessagesPlaceholder(variable_name='history'),
    ('human', '{input}'))
])
chain = prompt | llm | StrOutputParser()

store = {}
def get_session_history(session_id):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

with_memory = RunnableWithMessageHistory(
    chain, get_session_history,
    input_messages_key='input',
    history_messages_key='history'
)
config = {'configurable': {'session_id': 'user_1'}}
print(with_memory.invoke({'input': 'My name is Alex'}, config=config))
print(with_memory.invoke({'input': 'What is my name?'}, config=config))
```

### READ MORE — TOPIC REFERENCES

[Docs] LangChain — Managing chat history how-to

→ [https://python.langchain.com/docs/how\\_to/trim\\_messages/](https://python.langchain.com/docs/how_to/trim_messages/)

### ⑤ DAY COMPLETION CHECKLIST

- ☐ CLI chatbot remembers last 5 exchanges correctly
- ☐ RunnableWithMessageHistory set up with session\_id
- ☐ Memory persists across .invoke() calls within the same session
- ☐ Two different session\_ids have separate memories (tested)
- ☐ Summary memory kicks in after token threshold
- ☐ Verified: model correctly uses earlier context in a follow-up question

## Day 11 · Document Loaders & Text Splitters

### ① THEORY — TOPICS TO COVER

- Document loaders: PyPDFLoader, WebBaseLoader, CSVLoader
- RecursiveCharacterTextSplitter — how it works
- chunk\_size and chunk\_overlap — how to tune them
- Document metadata and how it flows through the pipeline

### ② FOCUS / SKIP GUIDE

### ◆ FOCUS ON

- How PyPDFLoader creates Document objects with page\_content and metadata
- How RecursiveCharacterTextSplitter tries separators in order
- Tuning chunk\_size — too small loses context, too large loses precision
- Adding source and page\_number metadata to track where chunks came from
- How metadata survives through the embedding and retrieval pipeline

### ✗ DON'T WORRY ABOUT YET

- SemanticChunker and other AI-based splitting — advanced topic
- Custom document loaders for proprietary formats
- Parallel loading of many documents at once

### RESOURCES — WATCH / READ BEFORE STARTING

- [Docs] LangChain document loaders reference  
→ [https://python.langchain.com/docs/integrations/document\\_loaders/](https://python.langchain.com/docs/integrations/document_loaders/)
- [Docs] LangChain text splitters guide  
→ [https://python.langchain.com/docs/concepts/text\\_splitters/](https://python.langchain.com/docs/concepts/text_splitters/)
- [Docs] LangChain — RecursiveCharacterTextSplitter  
→ [https://python.langchain.com/docs/how\\_to/recursive\\_text\\_splitter/](https://python.langchain.com/docs/how_to/recursive_text_splitter/)

### ③ PRACTICAL — TASKS TO COMPLETE

- ☐ Load a PDF with PyPDFLoader and inspect Document objects
- ☐ Load a webpage with WebBaseLoader
- ☐ Split docs with RecursiveCharacterTextSplitter — try 3 chunk sizes
- ☐ Add custom metadata to chunks (source, page number)

### TASK RESOURCES

- [Docs] LangChain — PyPDFLoader  
→ [https://python.langchain.com/docs/integrations/document\\_loaders/pypdfloader/](https://python.langchain.com/docs/integrations/document_loaders/pypdfloader/)
- [Docs] LangChain — WebBaseLoader  
→ [https://python.langchain.com/docs/integrations/document\\_loaders/web\\_base/](https://python.langchain.com/docs/integrations/document_loaders/web_base/)

### ④ CODE EXAMPLES

#### Load, split, add metadata

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Load
loader = PyPDFLoader('document.pdf')
docs = loader.load() # list of Document objects
print(docs[0].page_content[:200]) # text
print(docs[0].metadata)           # {source, page}

# Split
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=50,
    separators=['\n\n', '\n', '.', ' ', '']
)
chunks = splitter.split_documents(docs)

# Add custom metadata
for i, chunk in enumerate(chunks):
    chunk.metadata['chunk_id'] = i
    chunk.metadata['word_count'] = len(chunk.page_content.split())
print(f'Created {len(chunks)} chunks')
```

#### READ MORE — TOPIC REFERENCES

[Docs] LangChain — Full list of text splitters

→ [https://python.langchain.com/docs/concepts/text\\_splitters/](https://python.langchain.com/docs/concepts/text_splitters/)

#### ⑤ DAY COMPLETION CHECKLIST

- ☐ PyPDFLoader loads PDF — Document objects have page\_content and metadata
- ☐ WebBaseLoader loads a webpage successfully
- ☐ Tried 3 chunk sizes — noted how results differ
- ☐ Custom metadata (source, page number) added to all chunks
- ☐ Metadata survives chunking and is accessible later

## Day 12 · Vector Stores & Retrievers

#### ① THEORY — TOPICS TO COVER

- LangChain VectorStore interface: add\_documents, similarity\_search
- Chroma vs FAISS vs Pinecone — when to use each
- Retriever types: similarity, MMR, threshold
- MultiQueryRetriever — generating multiple search queries automatically

#### ② FOCUS / SKIP GUIDE

### ◆ FOCUS ON

- How `.as_retriever()` converts a vector store to a retriever
- `similarity_search` vs `similarity_search_with_score` — getting confidence
- MMR (Maximal Marginal Relevance) — diversity vs relevance tradeoff
- `search_kwargs={'k': 4}` — tuning how many chunks are retrieved
- When `MultiQueryRetriever` helps — when one query isn't enough

### ✗ DON'T WORRY ABOUT YET

- Self-query retriever and metadata filtering — Day 18 advanced topic
- Ensemble retriever with BM25 — Week 3 advanced topic
- Pinecone cloud setup and API keys — only needed at scale

### RESOURCES — WATCH / READ BEFORE STARTING

- [Docs] LangChain vector stores docs  
→ <https://python.langchain.com/docs/concepts/vectorstores/>
- [YouTube] Alejandro AO — LangChain + Chroma full tutorial  
→ <https://www.youtube.com/watch?v=3yPBVii7Ct0>
- [Docs] LangChain — Retriever types  
→ <https://python.langchain.com/docs/concepts/retrievers/>

### ③ PRACTICAL — TASKS TO COMPLETE

- ☐ Build a Chroma vector store from loaded PDF chunks
- ☐ Try `similarity_search` and `similarity_search_with_score`
- ☐ Create a retriever with `.as_retriever(search_kwargs={'k': 4})`
- ☐ Experiment with MMR retriever and compare result diversity

### TASK RESOURCES

- [Docs] LangChain — Chroma integration  
→ <https://python.langchain.com/docs/integrations/vectorstores/chroma/>
- [Docs] LangChain — MMR retrieval  
→ [https://python.langchain.com/docs/how\\_to/vectorstore\\_retriever/](https://python.langchain.com/docs/how_to/vectorstore_retriever/)

### ④ CODE EXAMPLES



#### Chroma vector store + retrievers

```
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model='text-embedding-3-small')
vectorstore = Chroma.from_documents(
    documents=chunks, embedding=embeddings,
    persist_directory='./chroma_langchain'
)

# Similarity search
results = vectorstore.similarity_search('What is RAG?', k=3)
results_with_score = vectorstore.similarity_search_with_score('RAG', k=3)

# Retriever
retriever = vectorstore.as_retriever(
    search_type='mmr',
    search_kwargs={'k': 4, 'fetch_k': 20, 'lambda_mult': 0.5}
)
docs = retriever.invoke('explain retrieval augmented generation')
```

#### READ MORE — TOPIC REFERENCES

**[Docs]** LangChain — MultiQueryRetriever

→ [https://python.langchain.com/docs/how\\_to/MultiQueryRetriever/](https://python.langchain.com/docs/how_to/MultiQueryRetriever/)

**[Blog]** Pinecone — FAISS vs Chroma vs Pinecone comparison

→ <https://www.pinecone.io/learn/series/faiss/>

#### ⑤ DAY COMPLETION CHECKLIST

- ☐ Chroma vector store built from PDF chunks
- ☐ similarity\_search returns relevant results
- ☐ similarity\_search\_with\_score returns scores (lower = better for Chroma)
- ☐ .as\_retriever() with search\_kwargs working
- ☐ MMR retriever tested — results are more diverse than similarity
- ☐ Understand when to use Chroma vs FAISS vs Pinecone

## Day 13-14 · Week 2 Project — Full RAG App

#### ① THEORY — TOPICS TO COVER

- RAG evaluation metrics: faithfulness, relevance, context recall
- LCEL RAG chain patterns in the official cookbook

#### ② FOCUS / SKIP GUIDE

### ◆ FOCUS ON

- Building a complete LCEL retrieval chain from scratch
- Adding conversation memory so follow-up questions work
- Including source citations using chunk metadata in answers
- Testing systematically — logging every failure
- The `create_retrieval_chain` helper vs manual chain construction

### ✗ DON'T WORRY ABOUT YET

- Automated evaluation with LangSmith — Day 17
- RAGAs evaluation library — Week 8
- Streaming responses in the UI — Week 7

### RESOURCES — WATCH / READ BEFORE STARTING

[YouTube] Alejandro AO — Conversational RAG with LangChain

→ <https://www.youtube.com/watch?v=LhnCsygAvzY>

[Docs] LangChain RAG tutorial (official)

→ <https://python.langchain.com/docs/tutorials/rag/>

[Docs] LangChain — Conversational RAG how-to

→ [https://python.langchain.com/docs/how\\_to/qa\\_chat\\_history\\_how\\_to/](https://python.langchain.com/docs/how_to/qa_chat_history_how_to/)

### ③ PRACTICAL — TASKS TO COMPLETE

- ☐ Build full RAG: load PDF → chunk → embed → store in Chroma
- ☐ Build LCEL retrieval chain: retriever | prompt | llm | parser
- ☐ Add conversation memory so follow-up questions work
- ☐ Test 20 questions and log failures/hallucinations
- ☐ Add source citations in answers using chunk metadata

### TASK RESOURCES

[Docs] LangChain — `create_retrieval_chain`

→ [https://python.langchain.com/docs/how\\_to/qa\\_chat\\_history\\_how\\_to/](https://python.langchain.com/docs/how_to/qa_chat_history_how_to/)

[Docs] LangChain — Passing context docs to chain

→ [https://python.langchain.com/docs/how\\_to/contextual\\_compression/](https://python.langchain.com/docs/how_to/contextual_compression/)

### ④ CODE EXAMPLES

#### Full RAG LCEL chain with citations

```
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain

system_prompt = '''Answer using only the context below.
Context: {context}
At the end, list sources as: Sources: [page X, page Y]'''

qa_prompt = ChatPromptTemplate.from_messages([
    ('system', system_prompt),
    MessagesPlaceholder('chat_history'),
    ('human', '{input}'))
])
question_answer_chain = create_stuff_documents_chain(llm, qa_prompt)
rag_chain = create_retrieval_chain(retriever, question_answer_chain)

response = rag_chain.invoke({
    'input': 'What is the main topic?',
    'chat_history': []
})
print(response['answer'])
print([d.metadata for d in response['context']]) # citations
```

#### READ MORE — TOPIC REFERENCES

**[Blog]** LangChain — RAG from scratch series (notebooks)

→ <https://github.com/langchain-ai/rag-from-scratch>

**[YouTube]** LangChain — RAG from scratch playlist

→ [https://www.youtube.com/playlist?list=PLfaIDFEXuae2LXbO1\\_PKyVJiQ23ZztA0x](https://www.youtube.com/playlist?list=PLfaIDFEXuae2LXbO1_PKyVJiQ23ZztA0x)

#### ⑤ DAY COMPLETION CHECKLIST

- ☐ Full RAG pipeline works end-to-end
- ☐ LCEL retrieval chain built and tested
- ☐ Memory added — follow-up questions use prior context
- ☐ Tested 20 questions — failures and hallucinations logged
- ☐ Source citations included in answers
- ☐ Ready to move to Week 3 — LangChain Advanced